Introduction

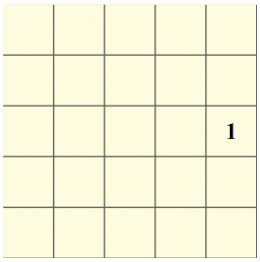
I programmed a graphical user interface application to play a number sequence memory game. With the Computer Programming course and the Programming Techniques and Application course, I've been taking inspiration from our labs and assignments to create this project. My user interface will be based with `Tkinter`. Here is how to play:

Game rules

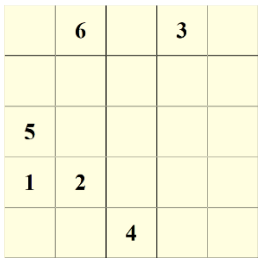
The user is faced with a squared board, having the same number of columns and same number of rows. The goal is to practice visual memorization.

When the game starts, the computer will randomly place a number from the sequence pattern in a square; the sequence pattern always starts with just [1].

Here are some examples for a 5 by 5 board:



Level 1: Number sequence [1]



Level 6: Number sequence [1,2,3,4,5,6]

The user must memorize the pattern on the screen. Once they're ready, they will click the numbers in increasing order, starting from 1. As soon as they click the first number, 1, the whole pattern will disappear from the screen.

The levels will continue to increase every time by adding one more number into the sequence pattern. Notice that the pattern will be randomized for every level-up, so the player must refresh the memory frequently and not rely on what they remembered from the previous level.

A win by the player is called when they finish the level corresponding to the number of squares on the board. For example, if they're playing the 5 by 5 board, the player wins if they manage to complete level 25.

The application is written in different files:

- `game.py` – It implements the class of the game with the rules and logic.
- `ui.py` – It implements the user interface.
- `btns_cls.py` – It implements certain classes necessary to be imported in both previous files.

Part I: `ui.py`

This file is what will initiate the whole game's interface. It will create the board (1000 x 1000) and create a starting page to introduce the user to the game (Figure 1).

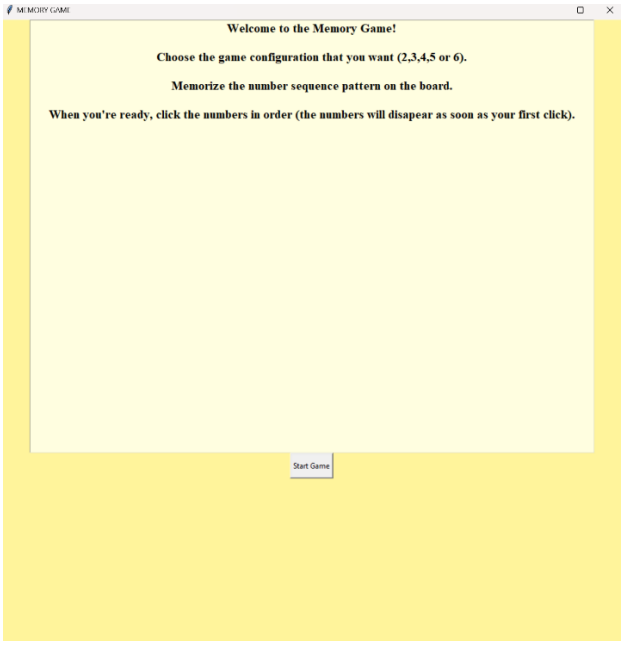


Figure 1

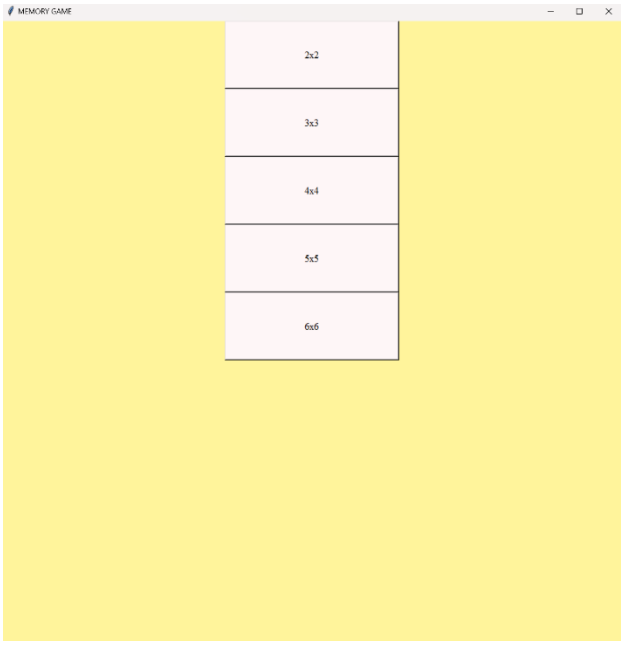


Figure 2

Once the user clicks the button "Start Game", the program will bring the user to choose which board configuration they would prefer to play, starting from a 2 by 2 board up to a 6 by 6 board (Figure 2). The more squares it has, the more difficult it would be to win the game.

Once the player has chosen the game configuration, the program will run the `game.py` file and start the game.

Part II: `btns_cls.py`

This file contains implementation of two different classes in order to create the buttons in `Tkinter`. The file first contains a global list variable that will contain all the configurations buttons (Figure 2).

1. `class btn_start()`

This class creates the buttons that would lead to the user to select the configuration, which is used for the "Start Game" (Figure 1) and the "Play Again" (Figure 3) buttons.

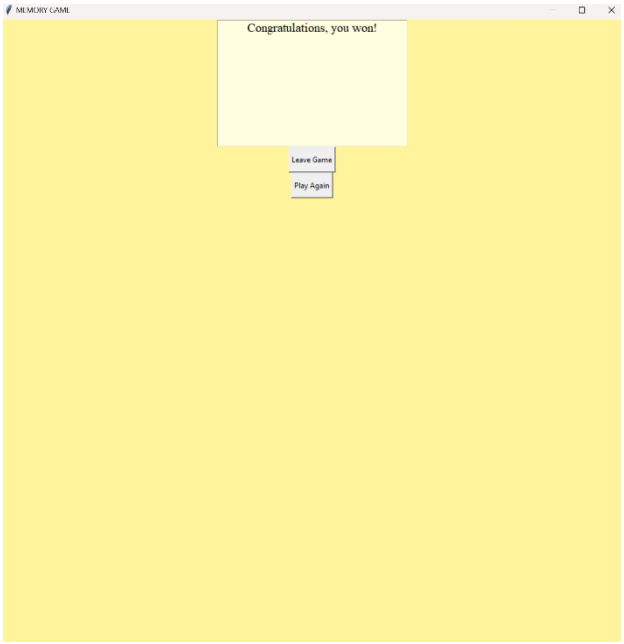


Figure 3

This class has the following methods:

- `def __init__(self, board, txt, g, notxt, link)`: The function creates the button on the board. It takes in five arguments:
  - The parent board of the button.
  - The text that goes on the button
  - The game class that it will initiate.
  - A text widget: this text widget is essentially linked to the button, such that when the button is clicked, the text will be destroyed with the button.
  - A list containing other buttons: these other buttons are also essentially linked to the created button, such that when the created button is clicked, the buttons in the list will be destroyed with the created button.
- `def c(self)`: The function is the command function that the start button will do. It will destroy itself, the text widget, and the buttons in the list (Note: the list can be empty). It then creates the configuration buttons (Figure 2) and adds them to the global list. The program allows 2x2, 3x3, 4x4, 5x5 and 6x6.

2. `class btn_level()`

This class creates the configuration buttons (Figure 2) that would let the user pick the number of squares total that he would be playing in the game.

This class has the following methods:

- `def __init__(self, board, txt, n, g)`: The function creates the configuration button on the board. It takes in five arguments:
  - The parent board of the button.
  - The text that goes on the button.
  - The number of rows/columns for the board (the configuration).
  - The game that it will initiate.
- `def destroy(self)`: The function will destroy the button itself.
- `def c(self)`: The function is the command function that the configuration button will do. It will destroy itself, and then destroy the other configuration buttons as well. It will then start the game with the appropriate board display.
- `def clicked(self)`: The function determines if the button has been clicked or not.

Part III: `game.py`

This file contains the most crucial part, as it runs the logic and the rules of the game. It only contains one class.

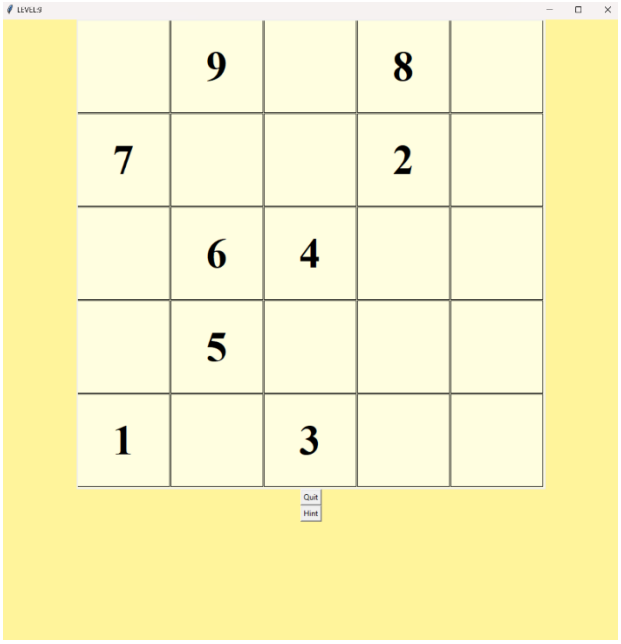


Figure 4

`class game()`

- `def __init__(self, board, n)`: The function will initialize the variables. It takes two arguments:
  - The parent board of the game.
  - The configuration that determines the number of rows/columns.

  The function computes the number of pixels by square it needs depending on the height and width of the canvas (750 x 750).

- `def start(self)`: The function will start the game by creating the canvas on the board with the necessary variables.

  It creates two buttons; "Quit" which will allow the user to quit the game, and "Hint" which will provide hints for the user.

  The function uses the graphic user interface to create the squares in the canvas and saves the IDs of the rectangles in a list. Using the function `def square(self, sqr)`, each square counting form left to right, from top to bottom, will have their bound coordinates saved in a dictionary according to the coordinate system of the canvas. (see the function below for more details)

  It then calls `def new_game` to create a new starting game.

- `def square(self, sqr)`: The function will do the necessary computations, such that given the "number" of the square position, it will return it's bound coordinates, which are two tuples, one for the x-axis and the other for the y-axis.

  Figure 4 demonstrates an example on how the "numbers" for the square position works for a 5 by 5 board configuration (Note: this has nothing to do with the number sequence pattern that the user will memorize, this is for a coordinate system of the board).
  Figure 5 shows how the canvas coordinate system works, and we're working with the x-axis going to 0 to 750 and the y-axis going from 0 to 750 given the dimensions of our canvas.

  Example: If you input the square number position 9, it will return (450, 600), (150, 300).

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Figure 5

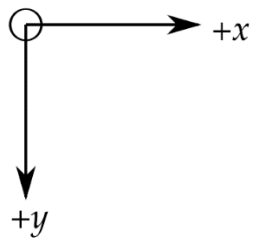


Figure 6

- `def new_game(self)`: The function will initiate the necessary values for the game logic when a new game starts. When the new game starts, the number sequence must begin with [1], so the user will begin with 0 correct clicks.

- `def memo(self)`: The function will give the game a randomized pattern for the user to memorizes. For every number in the number sequence, it will associate a random square number position to it. These combinations will be saved in a dictionary.

  Example: For the number 1 in the number sequence, it can randomly associate it to the square number position 17.

  Then, it called the `def num_to_pixel` to show the number on the appropriate canvas pixel coordinate system.

- `def num_to_pixel(self, num, sqr)`: The function will create the number sequence on the screen, it will place every number in the number sequence in the right place on the canvas pixel coordinate system. It takes two arguments, the number to put and the square position.

  It takes the bound coordinates of the square position, and it compute the (x, y) coordinate for the middle point of the square. It will then use canvas to create the appropriate number at that point.

  There are two dictionaries that stores important values:
  - IDS: Saves the ID of the created number on the canvas with the corresponding actual number.
  - PATTERN: Saves the number with the corresponding bounds coordinates of the square it is in.

- `def on_click(self)`: The function will handle any left click of the user. This is where the user is recreating the pattern from memorization.

  It first gets the (x, y) position of where the mouse clicked. It then follows this logic:
  1. Is the click within the canvas. If not, do nothing.
  2. Is the click corresponding to a position in the number sequence pattern (regardless of if it's the correct number in the pattern). If not, then the user doesn't have a valid click.
  3. Get the number from the sequence that it clicked.
  4. Is the number from the sequence the correct one in order. If not, then the user doesn't have a valid click (i.e. They clicked the number 5 in the pattern instead of the number 3 after succeeding 1 and 2).

  If their click is not correct, they lose the level, and goes back to level 1.

  If their click is correct, then we add 1 to number of correct clicks that they have. If the number of correct clicks that they have is equal to the length of the full number sequence, then they pass the level and can proceed to the next one, unless they've reached the maximum level already.

  When proceeding to the next level, we add one more number in the number sequence, but we reset the whole pattern on the canvas so that it regenerates a new randomized number sequence pattern to memorize. We call the `def memo` function.

  If they've reached the maximum level, they won the game.

- `def green_flash(self, num)`: The function will generate a green flash on the user's clicked square to indicate that they've clicked on the correct one.

  It takes the square number position and then gets the ID of the created square and changes the color fill to green for a quick time, then goes back to the original color.

- `def red_flash(self)`: The function will generate a red flash on the user's clicked square to indicate that they're clicked on the wrong one.

  It takes the (x, y) coordinates of the user's click and finds the associated number square position. Then, it gets the ID of the created square and changes the color fill to red for a quick time, then goes back to the original color.

- `def hints(self)`: The function is the command function of the "Hint" button. All the numbers of the sequence pattern will flash on screen for 0.3 seconds to freshen the user's memory.

  The user has 3 chances to use the hint button per game (not per level of the number pattern). i.e. They only have 3 chances for one full game configuration. Once they finished using the 3 chances, the button will be disabled. The number of chances that they have left will be displayed on the button.

- `def on_timer(self)`: The function will be used to handle a timer event. If the player is in the memorization state, the number pattern will be displayed for a maximum of 10 seconds.

  If the player does not click anything after 10 seconds, the number pattern will disappear, and the game will wait for the player to click on the first number.

  There is no time limit while the player is clicking to reproduce the number pattern, they can take as much time as they want to dig through their memory (no pressure 😊!).

- `def on_quit(self)`: The function is the command function of the "Quit" button. The game will close.

Once the user has won the game, it will generate the page on Figure 3, and let the user know that they won. The program will generate two buttons, "Play Again" which has the exact same function as the "Start Game" button (leading to the board configurations page) and "Leave Game" which has the exact same function as the "Quit" button (closing the game).